

UCS503P – Software Engineering
Nexora

An AI-Powered Security Event Correlation and Incident Reconstruction Platform

Mid-Semester Progress Report

Submitted By:

Apurv – 1024030791

Mohd. Noman – 1024030794

Shiven Bansal – 1024030799

Somraj Singha – 1024030795

Group- 3C55

Submitted To - Ms. Nisha Thakur



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Contents

Contents	2
Chapter 1: Introduction	3
1.1 Background and Context	3
1.1.1 Problem Statement	3
1.2 Project Objectives	3
Chapter 2: Methodology and Design	4
2.1 System Architecture.....	4
2.1.1 High-Level Design	4
2.1.2 Technical Stack	4
2.2 Implementation Details	5
2.2.1 Module 1: Core Functionality	5

Chapter 1: Introduction

1.1 Background and Context

As organisations instrument more of their infrastructure — logins, APIs, databases, file access, network calls — the volume of low-level anomaly alerts grows faster than any analyst team's capacity to triage them individually. A conventional anomaly detector treats every suspicious event in isolation: a credential attack, for example, typically unfolds as a sequence of repeated failed logins, an eventual successful login from an unfamiliar source, access to a sensitive resource, and an abnormal data transfer. Presented separately, these four events read as unrelated, low-context alerts, and the analyst is left to manually reconstruct the story behind them.

Correlation and incident-reconstruction capability of this kind already exists in mature commercial SIEM and XDR platforms, but it is either operationally heavy or priced for enterprise deployment, leaving smaller teams without an accessible equivalent. Nexora addresses this gap by pairing anomaly detection and attack classification with a correlation engine that reconstructs scattered suspicious events into a single, explainable security incident, at a scale and cost appropriate for a course-project setting rather than an enterprise SOC.

1.1.1 Problem Statement

Systems that flag anomalies without correlating them create four recurring problems:

- Alert fragmentation: a single coherent attack is reported as multiple disconnected alerts, forcing an analyst to manually infer the relationship between them.
- Loss of narrative context: individual alerts carry a risk score but no story — there is no indication of what an attacker was doing, only that something looked unusual at a point in time.
- Circular validation: systems trained and evaluated purely on self-generated simulated attacks cannot demonstrate that they generalise to real intrusion behaviour.
- High cost of the remedy: correlation and incident-reconstruction capability exists in mature commercial platforms, but is either operationally heavy or priced for enterprise deployment, leaving smaller teams without an accessible equivalent.

Nexora's core engineering problem is therefore twofold: build a detection and classification pipeline that is validated against real intrusion data rather than only against its own simulator, and build a correlation engine capable of reconstructing related events into one explainable incident.

1.2 Project Objectives

- Build and validate an anomaly-detection and attack-classification pipeline (Isolation Forest, with a supervised classifier for attack type) trained and benchmarked on a real, labelled intrusion dataset — CICIDS2017 or UNSW-NB15 — reporting precision, recall, and F1 per attack class, independently of the simulated demo environment.
- Design and implement the correlation and incident-reconstruction engine: an entity-linked event graph over a sliding time window, matched against a library of predefined attack-chain patterns, driven by a simulated organisational event pipeline (baseline traffic generator plus three to four attack injectors), targeting correct single-incident reconstruction in at least 90% of injected attack-chain scenarios.
- Deliver a three-tier system — FastAPI backend, PostgreSQL/Supabase storage, and a React + TypeScript dashboard updated over WebSockets — behind a CI/CD pipeline, so that a live attack-injection demo (detection → correlation → incident) is available by the final evaluation.

These objectives were chosen to be measurable in stages rather than all at once. At this stage, the event-schema data contract that all three objectives depend on is in place, along with the groundwork for the first: the real datasets have been acquired and explored, and the simulator's baseline traffic generator and initial attack injectors are functional. Model training and benchmarking against CICIDS2017/UNSW-NB15, the correlation engine itself, and the live end-to-end pipeline are scheduled for the remainder of the semester and will be reported at the final evaluation.

Chapter 2: Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Nexora is built as a three-tier architecture with a dedicated detection/correlation service, developed across four coordinated workstreams against a fixed data contract (event schema and incident schema) agreed in Week 1, so that the frontend and the backend/ML modules could be built in parallel rather than serially:

- Event pipeline: a simulated organisation generating login, API, database, file, and network events, with an injectable attack generator layered on top of normal baseline traffic.
- Detection engine: an Isolation Forest model that scores each event for anomalousness relative to learned normal behaviour, trained and validated on a real, labelled intrusion dataset rather than on simulator output alone.

- Classification engine: a supervised classifier (Scikit-learn) that labels a flagged event by attack type — brute-force / credential stuffing, port scanning, abnormal data exfiltration, or API request bursts.
- Correlation engine: an entity-linked event graph, built with NetworkX over a sliding time window, that links events sharing a user, IP, or session, and matches the resulting sequence against predefined attack-chain patterns.
- Dashboard: a React + TypeScript interface presenting a live activity stream, an anomaly-score graph, an incident timeline, an evidence panel, and a recommended action, communicating with the backend over REST and WebSockets.

This deliberately keeps the correlation engine a hybrid graph-linking and pattern-matching system rather than a learned causal-inference model — explainable, fully deterministic, and realistic to build and defend within a course timeline.

2.1.2 Technical Stack

Layer	Technology
ML / Data	Python; Scikit-learn (Isolation Forest, classifier); Pandas
Backend	FastAPI
Correlation	NetworkX (entity-linked event graph, attack-chain pattern matching)
Frontend	React + TypeScript; Recharts (anomaly-score chart); react-flow (incident timeline)
Database	PostgreSQL / Supabase
Real-time updates	WebSockets
Tools	Git/GitHub for version control; GitHub Actions for CI/CD

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

This module covers the functionality completed and demonstrable at the time of this submission.

Data contract. The team agreed and committed a shared event schema (event_type, timestamp, user, ip, session, severity) and incident schema (id, events[], severity, confidence, timeline, recommended_action) in Week 1. Every subsequent module — simulator output, correlation input, and dashboard mock data — is built directly against this contract, which is what has allowed the frontend and the data/ML workstreams to proceed in parallel.

Dataset acquisition and exploration. CICIDS2017 and UNSW-NB15 have been downloaded and profiled, including class balance and feature-list exploration on UNSW-NB15, as the groundwork for training and validating the Isolation Forest detector and the attack classifier against real intrusion data. Model training itself has not yet begun; this is the immediate next step for the detection workflow.

Simulated event pipeline. The baseline organisational traffic generator is functional, producing schema-compliant login, API, database, file, and network events for a small simulated organisation. On top of this baseline, one to two attack injectors are implemented — covering the auth-based attack type (brute-force / credential stuffing) and, where in progress, a second scoped attack type — so the simulator can already produce an injectable, ground-truth-labelled attack sequence for later use by the correlation engine and for detection testing.

Dashboard (mock-data phase). Several frontend components are built and rendering against mock JSON shaped exactly like the Week 1 schema, including the live activity feed and the anomaly-score chart, with further dashboard components (incident timeline, evidence panel) following the same schema-first approach. Because these components consume data matching the real contract, they are expected to require only a data-source swap — from mock JSON to the live FastAPI endpoints — once the backend is in place, rather than a rebuild.

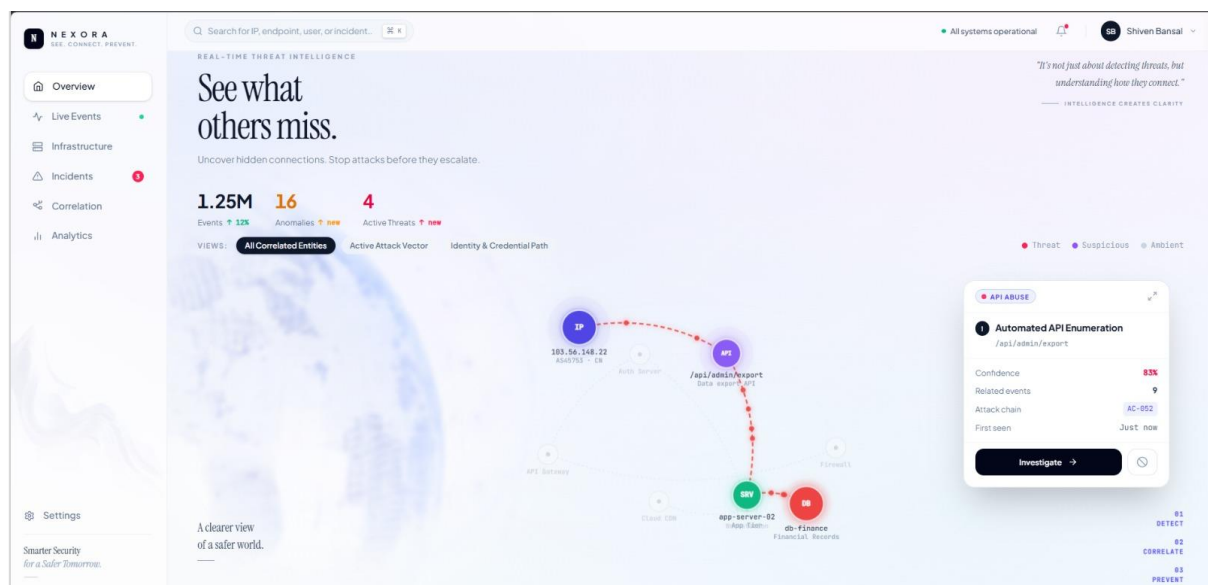


Figure 1: Nexora dashboard — correlated entity graph with a live "Automated API Enumeration" incident panel

The correlation engine, the FastAPI backend, and the WebSocket-driven end-to-end pipeline are scoped for the coming weeks and are not yet demonstrable. Together, the pieces completed so far establish the data foundation — a shared schema, a real validated data source in progress, and an injectable simulated event stream — that the detection, correlation, and integration work for the remainder of the semester will build directly on top of.